COMP 110

Time Complexity + “Big-O” Notation

Housekeeping

- EX09 Due Monday: *You can work alone if you want!*

Final Exam 4 pm April 30 in Hamilton 100

Make-up Final will be at 12pm on Friday, May 1 in SN014

- If you qualify for the makeup, *fill out the internal makeup request form ASAP*
- You won’t get a confirmation until next week

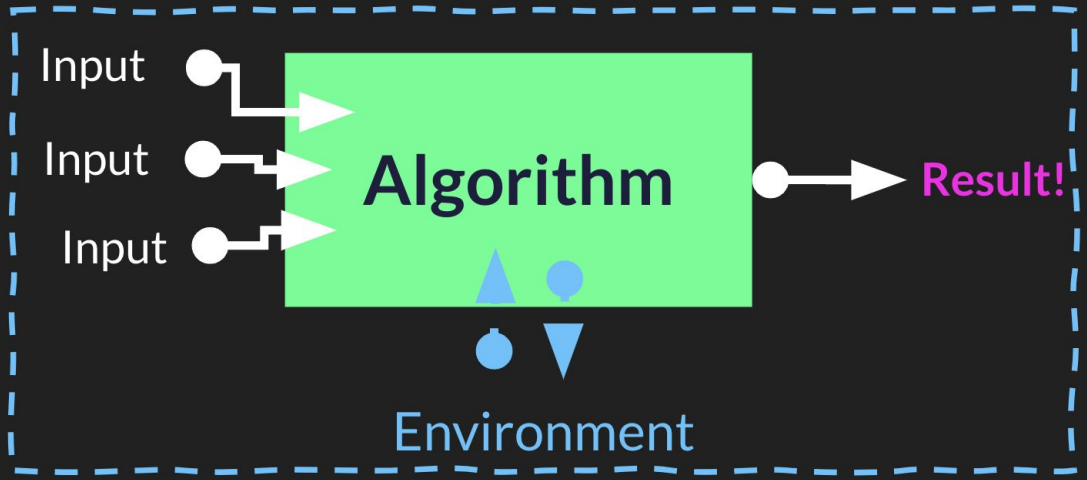
Recall: Algorithms

Input is data given to an algorithm

An **algorithm** is a series of steps

An algorithm **returns** some **result**

An algorithm *may* be influenced by its **environment** and it *may* produce side-effects which influence its environment.



How do we measure how “good” an algorithm is?

- How precise is it?
- How easy is it to implement?
- How much computer memory does it take?
- How *long* does it take to implement?

Is one factor *always* more important than the others, or would it vary by algorithm?

What kind of systems care about what factors more?

How do we measure how “good” an algorithm is?

- How precise is it?
- How easy is it to implement?
- How much computer memory does it take?
- How *long* does it take to implement?

All very important, but this is what we’ll discuss today!

Is one factor *always* more important than the others, or would it vary by algorithm?

What kind of systems care about what factors more?

Why do we care about computation speed?

- Security: Cryptography works because encrypted information takes *too long* to decipher!
- User Experience: Users don’t want to work with a slow application!
- Big Data: We want to be able to feed as much data as possible into our systems, but we need a way to *efficiently* do that!

Running time: how long does an algorithm take to run?

- Empirical analysis: write the code and test how long it takes to run!
  - Weaknesses:
    - You have to write the code for the whole algorithm and run it to see how long it will take
    - Different computers with different specs will have different runtimes
- Rather than using empirical analysis, computer scientists commonly consider the **number of operations (steps)** an algorithm requires
  - 1 operation == 1 step

Measurements We Use

Ω Best case (lower bound):

- Minimum number of operations (running time) required for the algorithm to execute

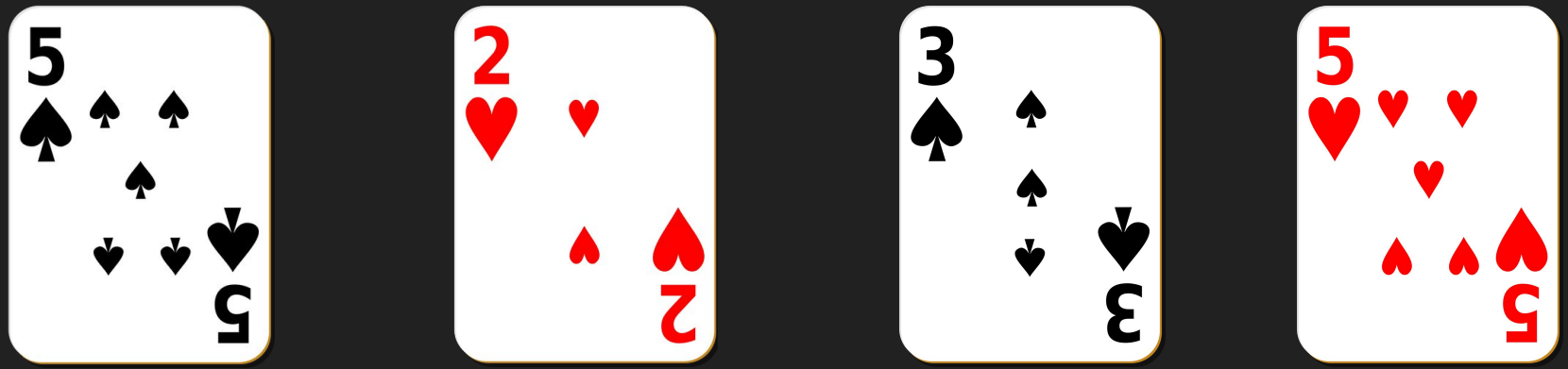
Θ Average case:

- Average running time among several different inputs

O Worst case (upper bound) ✨:

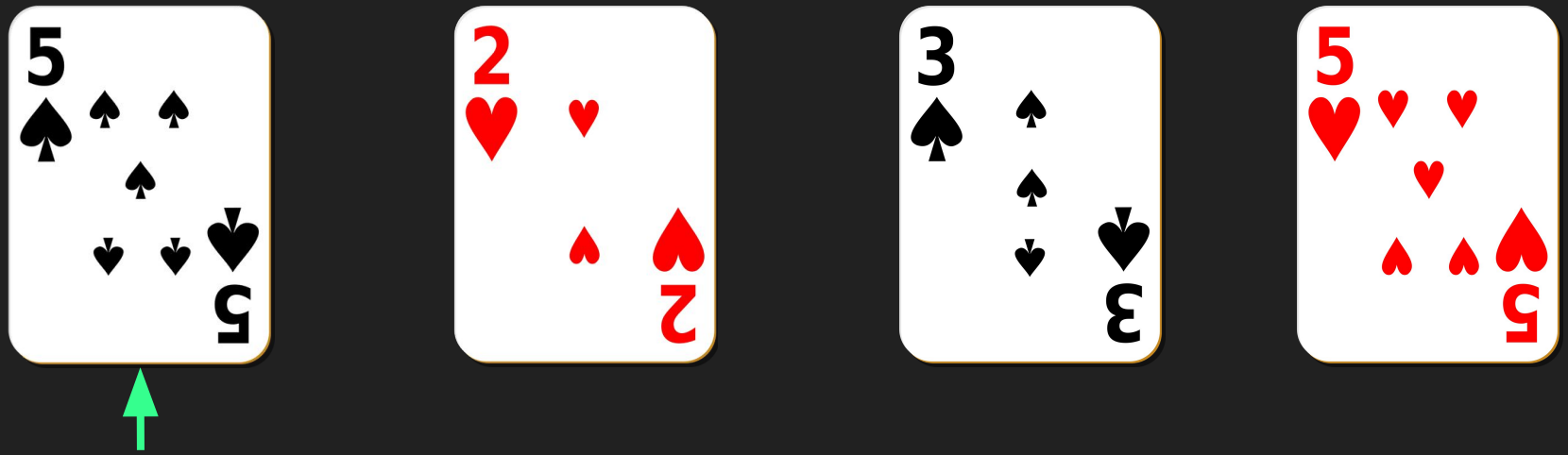
- The maximum running time given an input
  - How does the number of operations grow as an input grows?
- Important to understand how our algorithm will perform in the *worst* case
  - Prepare for the worst case. If an input ends up requiring fewer operations, great!!

Returning to Finding the Lowest Card in a Deck

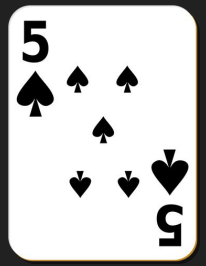


- Go from left to right
- Remember the lowest card you've seen *so far* and compare it to the next cards

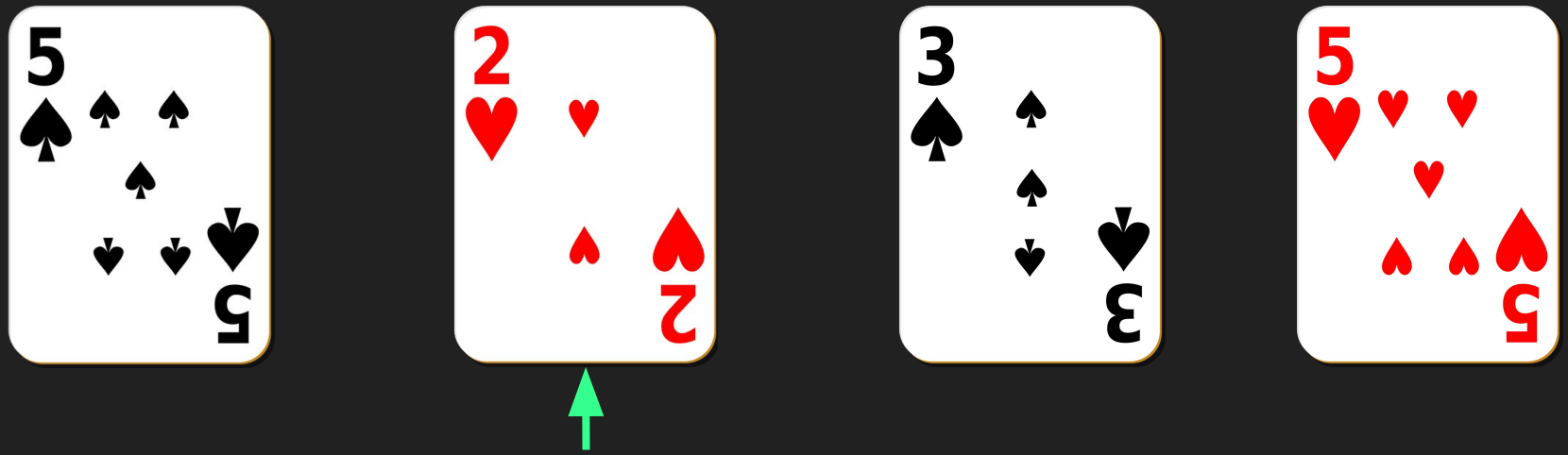
Finding the Lowest Card



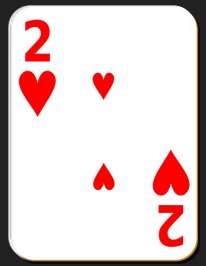
Low card:



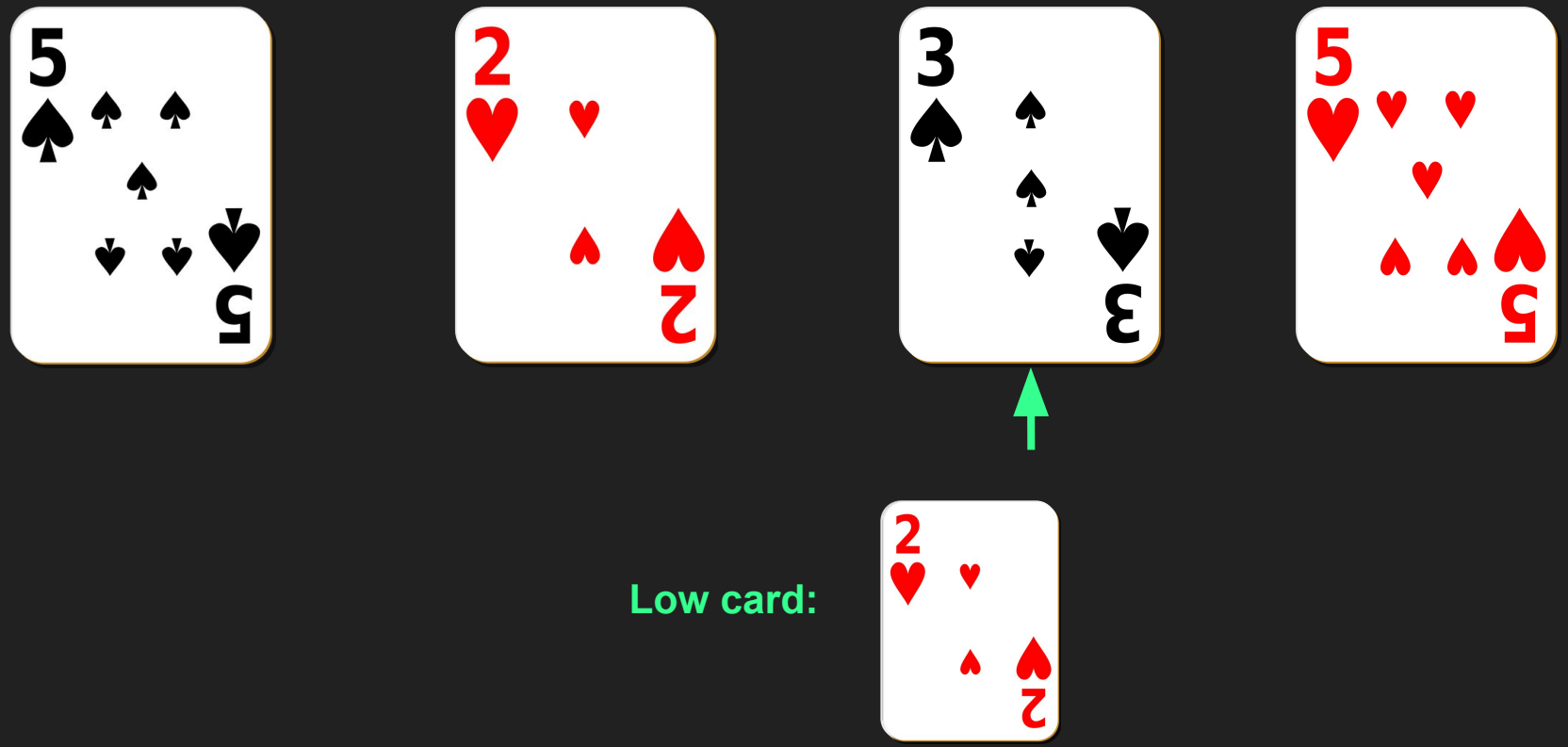
Finding the Lowest Card



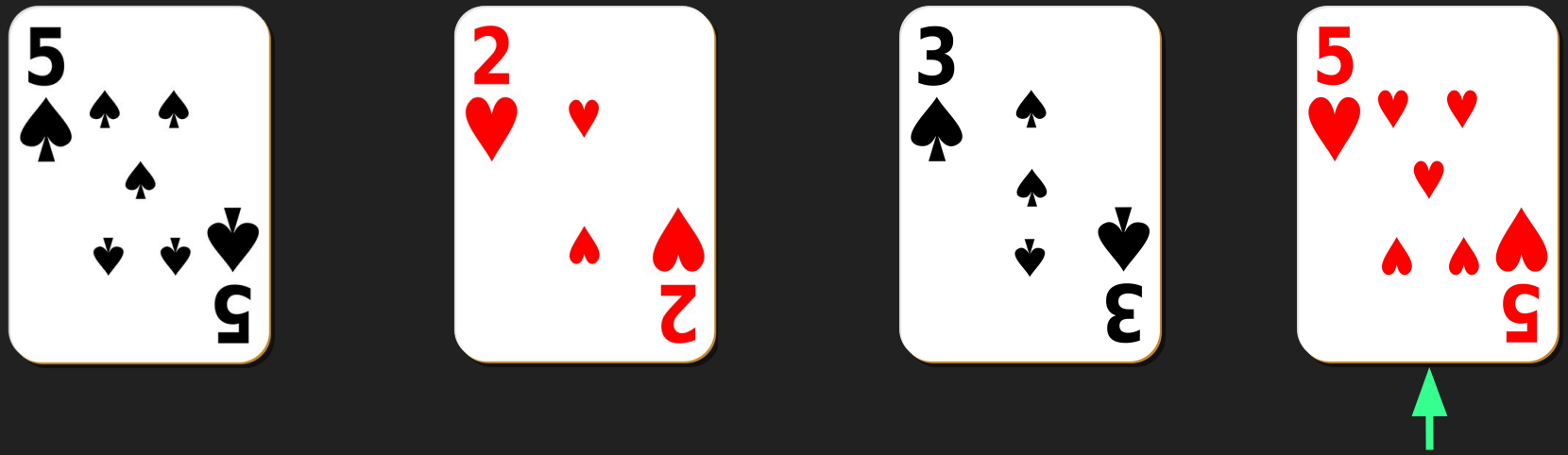
Low card:



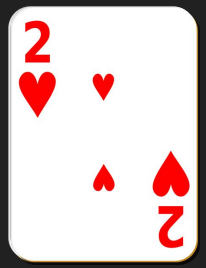
Finding the Lowest Card



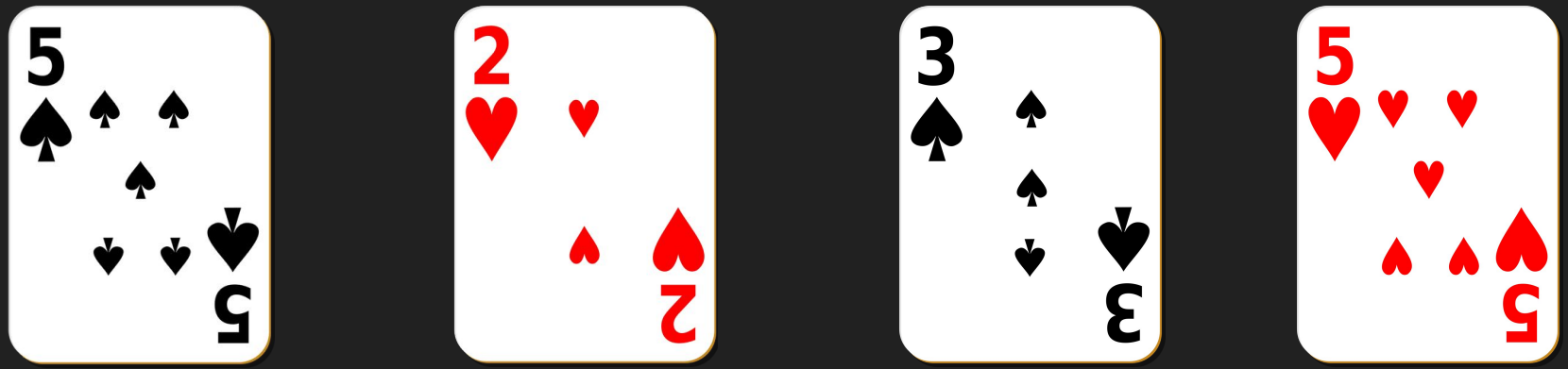
Finding the Lowest Card



Low card:

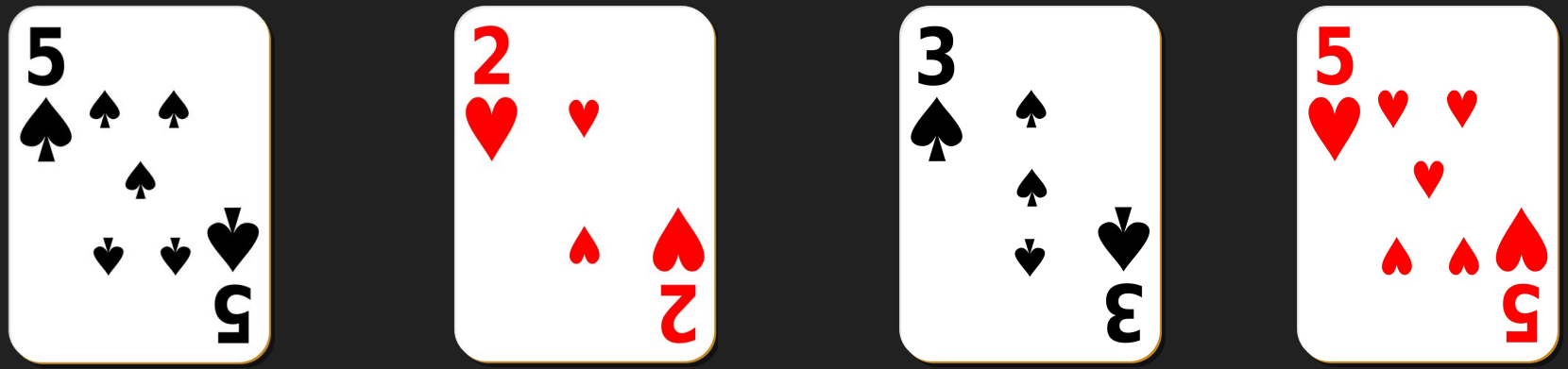


Finding the Lowest Card

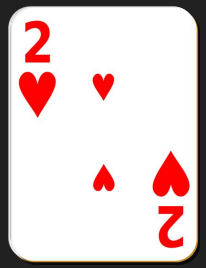


Low card:

Finding the Lowest Card

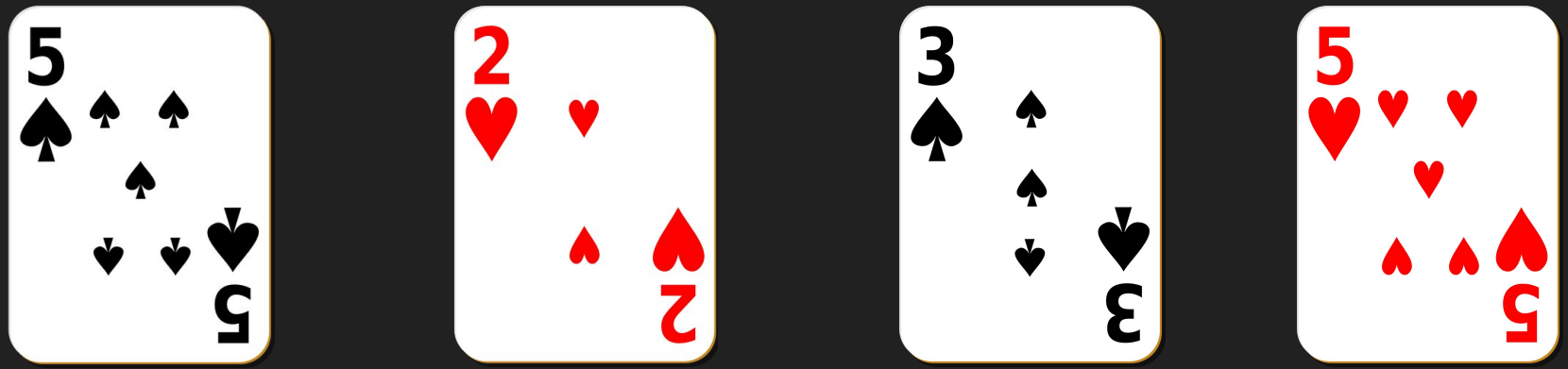


Low card:

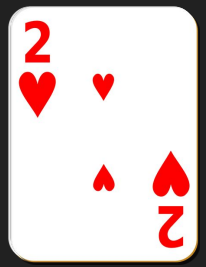


4 actions for input of 4 cards.

Finding the Lowest Card



Low card:



4 actions for
input of 4 cards.
→
n actions for
input of size n.

Finding the Lowest card

- In this approach, we always have to check every card in the deck, so our runtime will always be approximately *n* where *n* is the size of the deck.

$$\text{Finding the minimum} \in O(n)$$

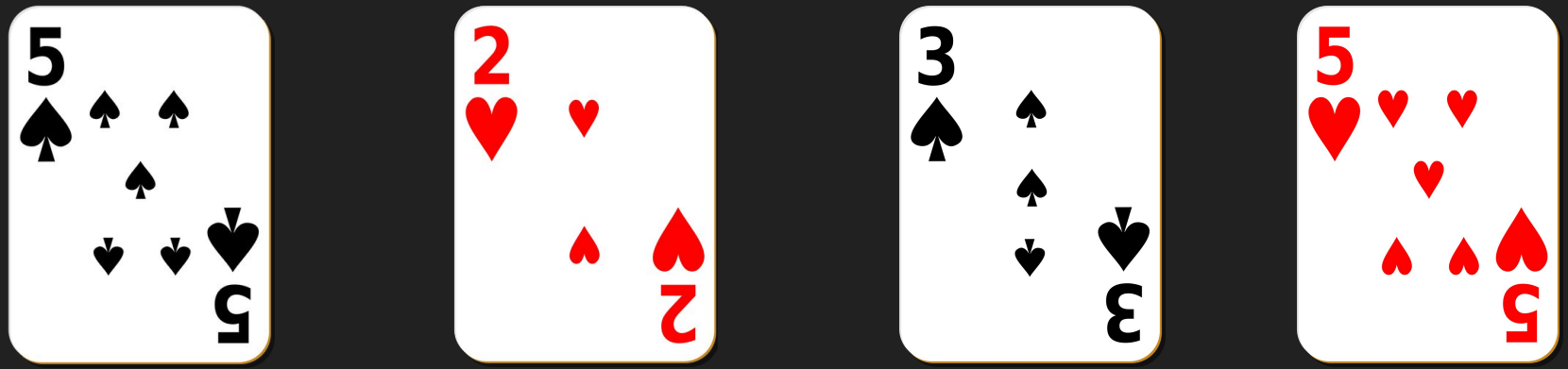
$$\text{Finding the minimum} \in \Omega(n)$$

$$\text{Finding the minimum} \in \Theta(n)$$

Speed vs. Memory

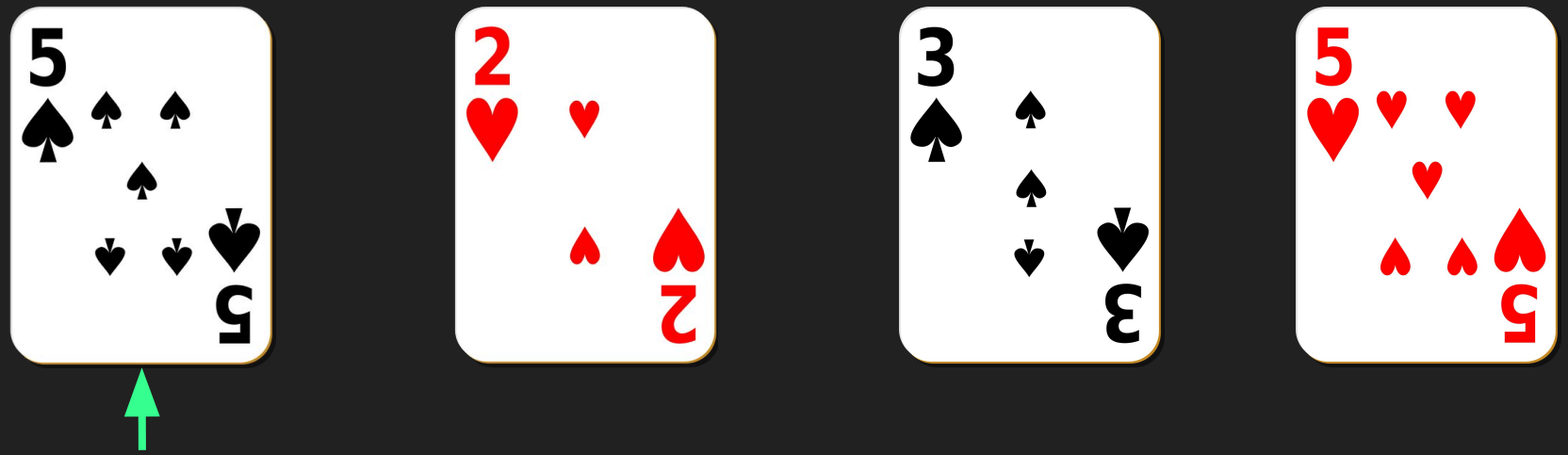
- Sometimes you can make a tradeoff between speed and memory.
- E.g. storing a value rather than computing it repeatedly.

New Example: Finding a specific card.

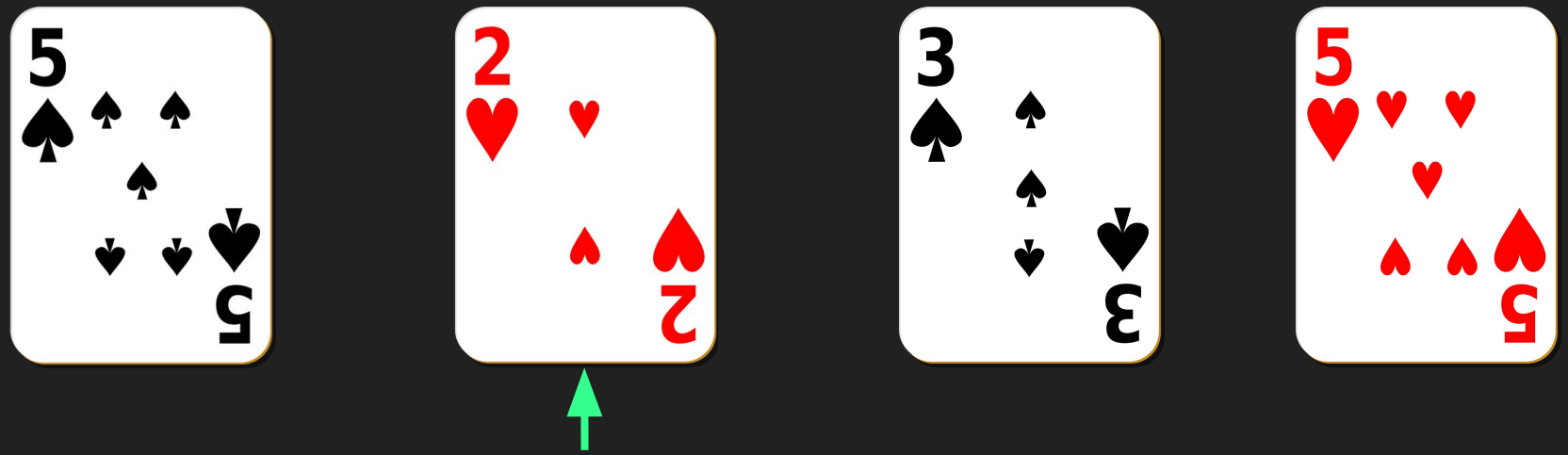


- Go from left to right
- The first time you see your card, exit!

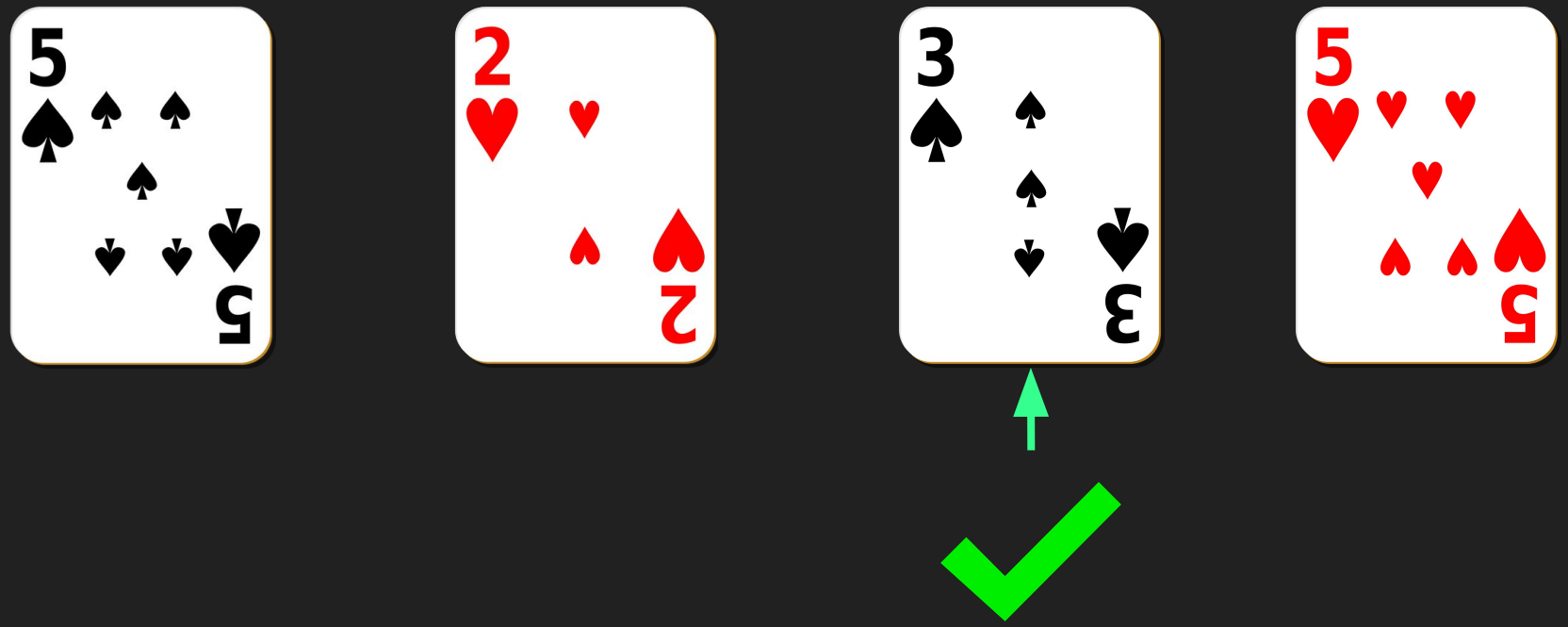
Finding 3



Finding 3



Finding 3

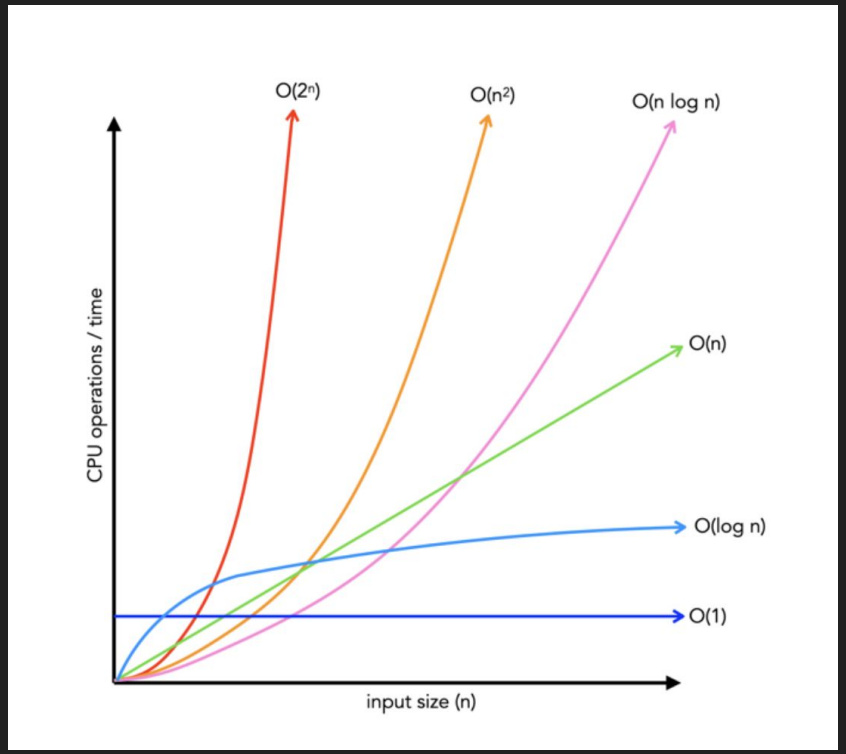


Discuss:

- What is the worst case input for this algorithm? (What will make us look at the *most* cards before exiting?)
- What is the best case input for this algorithm? (What will make us look at the *fewest* cards before exiting?)
- What is the Big-O (worst case) runtime in terms of deck size *n*?
- What is the best-case runtime in terms of deck size *n?*

Common Runtimes

- O(1) - Constant
- O(n) - Linear
- O(n^2) - Quadratic
- O(x^n) - Exponential (SLOW)



Source

Recall:

Data structure	Allows duplicates?	Ordered?	Fast lookups?	Fast Iterations?	Use Case
list [ ]	✅	✅	❌	✅ ✅	Ordered collections
set {}	❌	❌	✅	❌	Unique values, membership testing (fast lookups)
dictionary {key: value}	✅ *	It's complicated	✅	✅	Mappings, fast lookups, counting

List vs. Set Trade Off Example

```python
def intersection(a: list[str], b: list[str]) -> list[str]:
    result: list[str] = []

    idx_a: int = 0
    while idx_a < len(a):
        if a[idx_a] in b:
            result.append(a[idx_a])
        idx_a += 1

    return result
```

n operations

n operations

n*n operations

```python
def intersection(a: list[str], b: set[str]) -> set[str]:
    result: set[str] = set()

    idx_a: int = 0
    while idx_a < len(a):
        if a[idx_a] in b:
            result.add(a[idx_a])
        idx_a += 1

    return result
```

n operations

1 operation

n*1 operations

For simplicity, let's say the len of both **a** and **b** are **n**

List vs. Set Trade Off Example

```python
def intersection(a: list[str], b: list[str]) -> list[str]:
    result: list[str] = []

    idx_a: int = 0
    while idx_a < len(a):
        if a[idx_a] in b:
            result.append(a[idx_a])
        idx_a += 1

    return result
```

n operations

n operations

n*n operations

```python
def intersection(a: list[str], b: set[str]) -> set[str]:
    result: set[str] = set()

    idx_a: int = 0
    while idx_a < len(a):
        if a[idx_a] in b:
            result.add(a[idx_a])
        idx_a += 1

    return result
```

n operations

1 operation

n*1 operations

Suppose **a** and **b** each had 1,000,000 elements. The worst case difference here is approximately 1,000,000 operations, versus 1,000,000**2 or 1 trillion operations.

If your device can perform 100,000,000 operations per second, then...

This is a difference between 2.78 hours and 1/100th of a second.

Final Exam Review

Questions?